

Proposal: Replace Immediate Room Lock with a Provisional Hold + Auto-Expiry Booking Model

Prepared for: MIT

Author: Light Bekete

Date: September 2026

Status: For Review & Decision

1. Executive Summary

Today, tapping **Book Now** immediately marks a room as unavailable — before any payment is made. If the user never pays, the room stays locked to everyone else. This is an inventory leak that prevents paying customers from booking rooms that are, in reality, still empty.

This proposal replaces that model with a **Provisional Hold** — the pattern used by Booking.com, Softinn, Sirvoy, and AliExpress:

- The room is **held for a short window** (20 minutes) the moment the user taps Book Now.
- The hold **auto-expires** if payment does not arrive — the room returns to available.
- The room only becomes **permanently booked** when payment is confirmed.
- The data footprint is small: two extra fields per room, and one extra status value on the booking.

2. The Problem We Are Solving

2.1 Current Behavior (Broken)

Step	What happens	Result
1	User taps Book Now	Room marked unavailable immediately
2	User opens Payment screen	Room already locked for others
3a	Payment succeeds	Room stays unavailable (correct)
3b	Payment fails or user abandons	Room stays unavailable (leak)
3c	User never returns	Room remains locked indefinitely (worst case)

Figure 1 — Current flow: lock on Book Now, no reliable release path

2.2 Why This Is a Business Problem

Scenario	Consequence
User taps Book, fails payment	Room blocked for other paying customers
User taps Book, closes app	Room blocked indefinitely
User taps Book on 10 rooms "to try"	10 rooms blocked, zero revenue
Owner sees "unavailable" but room is empty	Cannot reconcile manually at scale

2.3 What Other Platforms Do

Booking.com — Temporary inventory block on booking request; released on failure or timeout.

Softinn — Holds inventory for a defined window, reminders before deadline, auto-cancel on expiry.

Sirvoy — Blocks the room for about 25 minutes while the guest pays; releases if payment does not complete.

AliExpress — Order awaits payment; inventory is not permanently removed until payment; order auto-closes after the window.

The pattern is universal: **hold, don't lock; expire, don't leak.**

3. Proposed Model: Provisional Hold

3.1 Core Principle

A room is "taken" only when it is either **actively held** by a non-expired hold or **confirmed booked**. The room remains available in the database unless a booking is truly confirmed.

3.2 Booking Status Lifecycle

When the user taps Book Now, a booking is created in **PENDING_PAYMENT** and a short hold is placed on the room. From that state only three outcomes are possible:

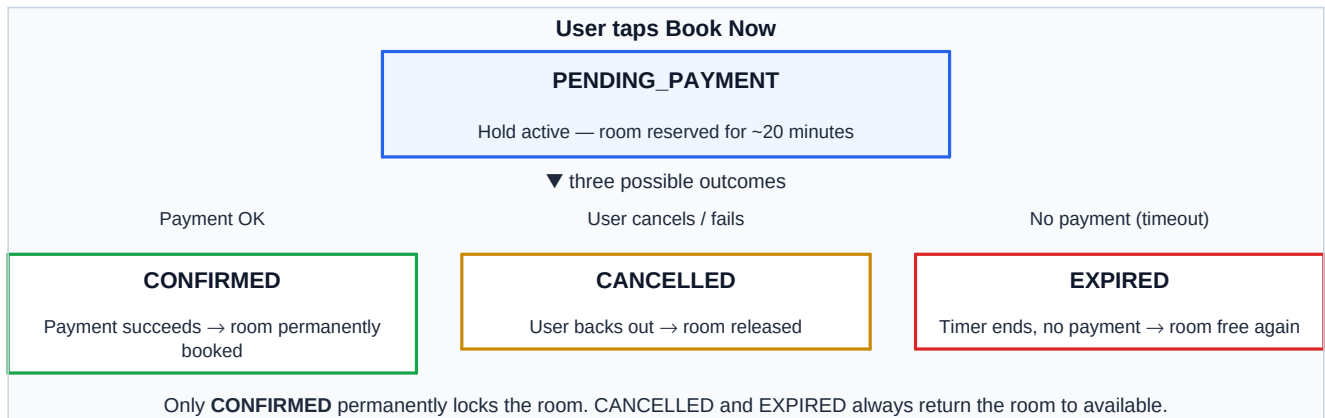


Figure 2 — Booking status lifecycle (kept on one view)

In short: every booking starts as a temporary hold. Payment success confirms it. Anything else (cancel, failed payment, or timeout) releases the room so another customer can book it.

3.3 What Changes on the Room

Each room keeps its existing fields. Two extra pieces of information are needed:

- **Current hold reference** — which booking (if any) currently holds the room, or empty if free.
- **Hold expiry time** — when that hold ends. After this moment the room is treated as free again.

The room's available flag stays true unless a booking is confirmed. While a hold is active, other users see the room as "On Hold" rather than permanently booked.

3.4 Booking Statuses

- **PENDING_PAYMENT** — hold is active; user is expected to pay within the window.
- **CONFIRMED** — payment succeeded; room is permanently booked.
- **EXPIRED** — hold timed out with no successful payment; room released.
- **CANCELLED** — user backed out or the system cancelled the attempt; room released.
- **FAILED** — a payment attempt failed; the hold can remain so the user may retry before expiry.

3.5 What Counts as a Bookable Room

A room is offered to new bookers only when it is marked available **and** either has no active hold, or its hold has already expired.

3.6 Before vs After

Aspect	Before (current)	After (proposed)
On Book Now	Room locked immediately	Short provisional hold started
If user abandons	Room stays locked	Hold expires → room free
If payment fails	Room stays locked	Hold continues until expiry; user may retry
If payment succeeds	Room unavailable (correct)	Confirmed → room permanently booked
Inventory leak	Yes — unbounded	No — timer-bounded
Audit trail	Weak	Full status history

Figure 3 — Side-by-side comparison

4. User Experience

4.1 For the Booking User

- **Tap Book Now** — room held for 20 minutes; complete payment to confirm.
- **Payment screen** — live countdown of remaining hold time.
- **Payment succeeds** — booking confirmed; room permanently booked.
- **Payment fails** — hold still active for a limited time; user can retry.
- **User backs out or time runs out** — hold ends; room available again.

4.2 For Other Users

- Rooms under an active hold show as **"On Hold"** (not fully booked).

- When the hold expires, the room shows as **"Available"** again.
- Later (optional): "Notify me if this room frees up."

4.3 Notifications (recommended)

Trigger	Channel	Message intent
Hold created	In-app	Room held for a limited time — complete payment
Few minutes left	Push/SMS (optional)	Reminder that the hold is about to expire
Hold expired	Push/email	Hold ended; room is free again
Payment confirmed	Push/email	Booking confirmed

5. Why This Design

5.1 Why a Timer Instead of "Reset on Failure" Alone?

Failure is often an absence of events: the phone dies, the app is closed, or the user walks away. A fixed hold window is the only reliable way to free inventory without waiting for a signal that might never come.

5.2 Why Keep Expired Bookings Instead of Deleting Them?

Deleting a hold removes the audit trail. Late or delayed payment confirmations need a record so support can reconcile. Keeping EXPIRED or FAILED bookings makes that possible.

5.3 Why a Scheduled Release Job?

We need a controlled transition (pending → expired), clear hold fields, and optional notifications — not silent deletion. A regular background job gives control, logging, and one place to observe and debug.

5.4 Data Footprint

- Two small fields on each room (who holds it, and until when).
- A few extra status values on bookings.
- No new collections required for the core design.

5.5 Concurrent Book Attempts

Two users must not both receive a hold on the same room at the same moment. Claiming the hold must be atomic: only one request succeeds; the other is told the room is currently on hold.

6. End-to-End Behaviour (What Should Happen)

6.1 Starting a booking

On Book Now, create a PENDING_PAYMENT booking and place a time-limited hold if the room is free. If the room is already held, reject the attempt. If accepted, return the expiry time so the app can show a countdown.

6.2 Successful payment

On payment confirmation, set booking to CONFIRMED, mark the room permanently unavailable, clear temporary hold fields, and notify the user.

6.3 Failed payment

Mark the booking FAILED if needed, but keep the hold until the window ends so the user can retry. Do not permanently lock the room.

6.4 Abandonment or timeout

When the hold window ends without successful payment, expire the booking, clear the room hold, and make the room bookable again. Optionally notify the user.

6.5 Listing rooms for other users

Only offer rooms that are available and either have no hold or whose hold has expired. Active holds show as "On Hold".

6.6 Client countdown

Show remaining hold time on the payment screen. When time reaches zero, disable payment and explain that the hold has expired.

#	Actor	Action	What should happen
1	User	Taps Book Now	Create pending booking; place time-limited hold if room free
2	App	Opens payment	Show countdown from hold expiry
3a	Payment	Success	Confirm booking; permanently book room

#	Actor	Action	What should happen
3b	Payment	Failure	Mark failed; keep hold until window ends
3c	User	Leaves / no pay	Hold remains until expiry
4	System	Hold time ends	Expire booking; clear hold; room free again
5	Others	Browse rooms	Only free or expired-hold rooms are bookable

Figure 4 — End-to-end behaviour

7. Performance & Scale

Concern	Assessment
Extra room data	Two fields per room — negligible
Extra booking statuses	A few values — negligible
New collections	None required for the core design
Background release	Light periodic check of expired holds only
Concurrent holds	Atomic claim so only one holder wins per room
Listing rooms	Available and (no hold or hold expired)

8. Migration Plan

8.1 Schema and status changes

Add hold reference and expiry on rooms; extend booking statuses; index rooms so bookable queries stay fast.

8.2 Existing data

Existing rooms get empty hold fields. Existing in-progress bookings are reviewed case by case.

8.3 Rollout phases

Phase	What	Risk
1	Add hold fields, statuses, and background release	Low — additive
2	App countdown and expiry handling	Low — UI
3	New flow for new bookings; monitor metrics	Medium
4	Remove old immediate-lock behaviour after stable period	Low
5	Optional waitlist / notify-when-free	Nice-to-have

8.4 Metrics to watch

- How many holds are created.
- Share of holds that become confirmed bookings.
- Share that expire without payment.
- How often users hit "room on hold".
- Typical time from hold to payment (to tune the window).

9. Open Questions for the Team

1. **Hold duration** — Is 20 minutes enough, or do mobile-money flows need longer?
2. **Payment provider latency** — Should "payment in flight" be a special non-expiring state if confirms arrive late?
3. **Multiple holds per user** — Cap simultaneous holds (e.g. max three)? Recommended: yes.
4. **Waitlist** — Build "notify me when free" now or later?
5. **Admin visibility** — Live "currently held" view for admins?

10. Recommendation

Adopt the **Provisional Hold + Auto-Expiry** model.

- Stops inventory leak without forcing users to finish abandoned flows.
- Small data changes on rooms and bookings, plus a simple background release step.
- Concurrent book attempts stay safe with atomic hold claims.
- Status history supports support and reconciliation.
- Paying customers can book rooms that would otherwise stay locked.

Approve and begin with Phase 1: hold fields, statuses, and automatic release of expired holds.

Attachments for the review meeting:

- Current flow (Figure 1)
- Proposed lifecycle (Figure 2)
- Before vs after (Figure 3)
- End-to-end behaviour (Figure 4)